

Exercice 1 : La classe `ToDoList`

Une liste de tâches est représentée par un tableau qui peut initialement contenir un maximum de cinq tâches. Au besoin, la structure de données peut être agrandie de 5 cases à la fois. La gestion de l'ajout et de la suppression des tâches se fait de manière particulière : une tâche est ajoutée dans la première case vide trouvée alors que la suppression revient à remplacer une tâche stockée par la valeur `null`. Les tâches ne sont donc pas forcément stockées dans des cases contiguës.

Q1. Créez la classe `ToDoList`, munie d'un constructeur sans paramètre et de la méthode `enlarge`, permettant l'augmentation du nombre de tâches en attente.

ToDoList
- chores : Task[]
+ ToDoList()
+ enlarge() : void

Q2. Écrivez une méthode `toString` qui retourne l'objet sous forme textuelle :

```
String toString()
```

Q3. Écrivez la méthode `addTask` qui permet d'ajouter une tâche dans la première case libre identifiée.

```
void addTask(Task aTask)
```

Q4. On souhaite pouvoir supprimer une tâche soit en fournissant la tâche elle-même, soit en fournissant son indice. Écrivez les méthodes nécessaires pour cela.

```
void removeTask(Task aTask)
```

```
void removeTask(int i)
```

Q5. Écrivez une méthode `isOverwhelmed` qui détermine si une liste de tâches est pleine.

```
boolean isOverwhelmed()
```

Q6. Écrivez une méthode `getNbTask` qui détermine le nombre de tâches présentes dans la liste.

```
int getNbTask()
```

Q7. Écrivez une méthode `onSickLeave` qui repousse toutes les tâches de la liste d'un nombre de jours donné.

```
void onSickLeave(int nbDays)
```

Q8. Écrivez une méthode `dueTasks` qui retourne la liste des tâches arrivant à échéance ce jour.

```
Task[] dueTasks()
```

Q9. On souhaite pouvoir trier notre liste de tâches selon différents critères comme l'ancienneté, l'urgence ou la durée. Écrire les fonctions nécessaires à ces tris.

```
void emergencySort()
```

```
void durationSort()
```

```
void senioritySort()
```

Q10. Créez une classe `UseToDoList` munie d'une méthode `main` réalisant le scénario suivant :

1. Création de deux listes nommées "*Alice*" et "*Bruno*";
2. Ajout de 3 tâches à *Alice* dont les dates limites arrivent à échéance dans 3, 4 et 5 jours et dont les descriptions sont respectivement *a1*, *a2* et *a3*;

3. Ajout de 3 tâches à *Bruno* dont les dates limites arrivent à échéance dans 1, 5 et 9 jours et dont les descriptions sont respectivement *b1*, *b2* et *b3* ;
4. Affichage de leur liste de tâches respectives ;
5. *Bruno* se fait porter pâle pour une période indéterminée, *Alice* hérite donc de son travail ;
6. Affichage de leur liste de tâches respectives ;
7. *Alice* obtient le report de sa tâche la plus longue et de sa tâche la plus ancienne ;
8. Affichage la liste de tâche d'*Alice*.

Q11. Réalisez un `commit` de votre travail associé au message : `"tp00-03::exo-ToDoList"`. Soumettez l'ensemble des modifications réalisées à votre dépôt `gitlab`.